

Computer Architecture Container Workbench

Building It – and the Real Snags Along the Way

Where does the machine end?

A dedicated Linux workbench, built the hard way, so you don't have to hit the same snags.

Companion slides for the recorded walkthrough – every gray box below is meant to be copied straight into your terminal or pasted into a Copilot/ChatGPT chat.

Why a second container, just for Architecture?

- ▶ The shared Commons container proved the *loop*: build, publish, pull elsewhere.
- ▶ Architecture's own question is different: **where does the machine end?** What does the container hold constant, and what still leaks in from the host?
- ▶ This workbench adds the tools Architecture actually uses – gcc, objdump, readelf, strace – inside a container you can compare against your own machine.

This is optional enrichment

It does not replace archprobe and does not change what you must submit for Week 3.

Snag #1: a pasted script that looks right but isn't

COPY/PASTE THIS – what actually broke

```
$ErrorActionPreference = 'Continue'  
$EvidenceDirectory = Join-Path $Week3Directory 'evidence'
```

What happened

The pasted text looked fine on screen, but the quote characters and dash had been silently swapped for "smart" typographic look-alikes. PowerShell read *Continue* and *Join-Path* as garbage and rejected them.

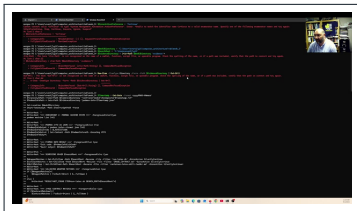
The fix

Paste code as **plain text** (paste, then undo-formatting – or paste into Notepad first). An error naming something that looks *almost* right means: suspect smart quotes/dashes first.

Recovery: confirm Podman is actually ready

COPY/PASTE THIS – run this before anything else

```
podman machine list  
podman info
```



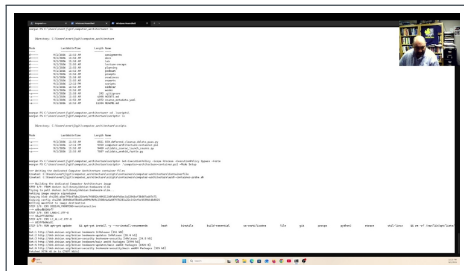
Why this step, every time

Do not troubleshoot a build failure before you know Podman itself is healthy. If `podman info` prints real output instead of a connection error, the machine is ready and the problem is somewhere else.

The real setup, once the paste is clean

COPY/PASTE THIS – one command, from the repository root

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force  
.\scripts\computer-architecture-container.ps1 -Mode Setup
```



What Setup does

Writes the Containerfile and probe script, builds the image, and runs the probe once – all from one clean command, once the earlier paste problem was gone.

Publish it: push to GHCR

COPY/PASTE THIS – after Setup succeeds

```
.\scripts\computer-architecture-container.ps1 -Mode Push
```

What you'll be asked for

A **classic** GitHub personal access token scoped to write:packages, pasted at the hidden password prompt – never your real GitHub password.

One manual step after the push

New GitHub packages are private by default. To let anyone pull without logging in: GitHub profile → Packages → the package → Package settings → Change visibility → Public.

Or skip the build entirely – pull the published image

COPY/PASTE THIS – fastest path, no build required

```
podman pull ghcr.io/jeremy-evert/computer-architecture-lab:week03
podman run -rm -volume "$PWD:/workspace:Z" -workdir /workspace \
ghcr.io/jeremy-evert/computer-architecture-lab:week03 arch-container-probe
```

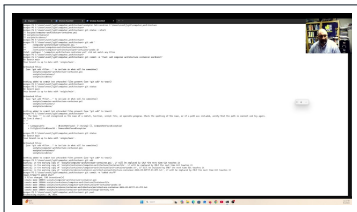
Using Docker instead of Podman?

Same commands, docker instead of podman.

Wire it into the course

COPY/PASTE THIS – save your work the same way every time

```
git add scripts
git commit -m "Add computer architecture container workbench"
git push
```



What just got saved

The Containerfile, the probe script, the PowerShell driver, and the evidence receipts – committed and pushed to the course repository, so the next person (including future you) does not have to rebuild this from memory.

Where to go from here

- ▶ Full command reference and troubleshooting:
`scripts/containers/computer-architecture/README.md`.
- ▶ Full recorded walkthrough: Computer Architecture Week 3 module, Canvas.
- ▶ Compare this container's receipt against your own machine's `archprobe` receipts from earlier in the week – that comparison *is* the Architecture lens on this exercise.

The one-sentence takeaway

A container can hold the software environment constant. It cannot hide the CPU, the kernel, or the host underneath it – and knowing which is which is the whole point.